

Fundamentals of Robotics Project

Sara Francavilla & Nivriti Raisingh

7 June 2025

Abstract

The project equips a UR5 robot with the ability to detect objects, plan collision-free motions, and execute grasp-and-place manoeuvres in real time. All software is written for ROS 2 Humble and runs on a single Linux workstation. Although the pipeline functions reliably from perception to trajectory synthesis, we have observed sporadic goal-rejection errors at the very last boundary where the generated `JointTrajectory` is handed to the hardware controller. Careful analysis revealed that the problem arises from clock discrepancies introduced when RViz 2 operates in simulated-time mode; the controller discards goals whose header stamps appear to lie in the past. This limitation does not affect algorithmic validity, yet it reminds us that representation and synchronisation issues can undermine an otherwise sound architecture.

1 The Workflow

When the system boots, the RGB-D camera mounted above the workspace begins streaming colour and depth images. A set of tightly coupled ROS 2 nodes turns those raw pixels into motor torques in a matter of hundreds of milliseconds, following a deterministic sequence. First, an image reaches the *vision package*, where a convolutional neural network determines the presence and location of target objects. Immediately afterwards, depth data are employed to lift the 2-D detections into metric 3-D space expressed in the robot's base frame. The resulting object pose serves as the cue for the *planning package*. A finite-state machine running therein selects the next manipulation strategy, for example, “approach”, “grasp”, or “place”. For each stage the planner issues two Cartesian poses—a starting pose and a goal pose—to the *motion package*. Inside that package the poses are interpolated into a dense path, converted into joint space through inverse kinematics, and time-parameterised so that every joint moves smoothly within its velocity and acceleration limits. The output is a fully fledged `JointTrajectory` that the *interface package* forwards to the UR5 controller. During execution the planner monitors feedback and schedules the next action, thereby closing the loop until all objects have been processed.

2 Vision Package

The project utilizes a fine-tuned YOLOv11 model trained on a dataset provided via the course's Moodle page. The dataset was split into training, validation, and testing subsets. Three YOLOv11 model variants of different sizes were trained and exported as C++-compatible TorchScript files. The **small** architecture was selected for deployment due to its superior accuracy compared to the nano variant and faster inference time relative to larger models. This version achieved a precision of 86.93% and a recall of 76.35%.

The vision package initiates the operation, aiming to convert sensor data into a geometrically accurate scene description. Each incoming RGB frame is resized and normalized in the `detection.cpp` node before being passed to the convolutional neural network (CNN). The network outputs bounding box coordinates, dimensions, and class confidence scores. Detections with confidence above 70% are selected and republished as a plain, header-free array containing the class ID and normalized center coordinates of each bounding box.

Upon receiving a detection message on the `inference_result` topic, the node `position.cpp` awakens for a brief interval. It fetches the corresponding depth point cloud, projects the 2-D bounding box into the cloud's coordinate system, and yields a three-dimensional centroid. This centroid is then transformed into

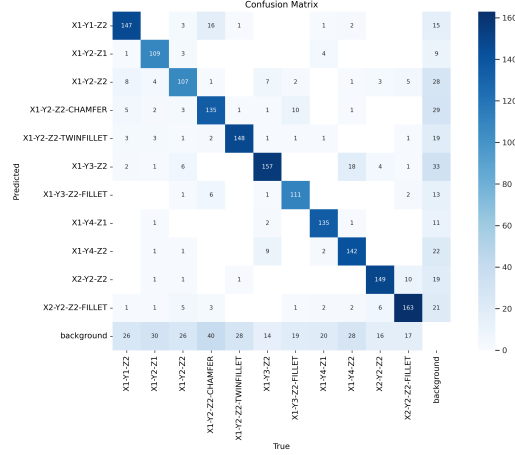


Figure 1: Confusion matrix of YOLOv11 fine-tuned on a custom dataset, showing high accuracy with some class overlap.

the `base_link` frame by querying the TF buffer. The process culminates in the publication of a `ClassPose` message, encapsulating the object’s position, a default identity orientation (in this case identity, because no orientation can be inferred from a single centroid), and its class label, marking the conclusion of the vision stage.

3 Planning Package

The planning package assumes responsibility as soon as a fresh `ClassPose` arrives. At its heart lies `control_unit.cpp`, a finite-state machine that cycles through the canonical manipulation states: *approach*, *pre-grasp*, *grasp*, *retreat*, and *place*. In the *approach* state the node constructs a pose directly above the object, leaving a configurable clearance along the positive *z*-axis. A second pose, offset downward by the object’s height plus a safety margin, embodies the *pre-grasp* goal. Both poses form a `StartEndPosition` structure that the planner publishes. When feedback signals that the arm has reached the pre-grasp pose, the state machine issues a service request to close the gripper, waits for confirmation, lowers the arm by a few millimetres into the exact *grasp* pose, and so forth until the object rests at its destination. At each transition the planner republishes the next pair of poses, ensuring that downstream nodes are triggered in the proper sequence.

4 Motion Package

Receiving a `StartEndPosition` message prompts the motion package to perform three consecutive computations. The first step unfolds inside `compute_path.cpp`, where a simple yet effective SLERP-based interpolator expands the sparse pair of poses into a dense Cartesian path containing one hundred and one waypoints. Directly afterwards, `compute_trajectory.cpp` subscribes to the path, interrogates the present joint configuration via `/joint_states`, and launches a vectorised inverse-kinematics solver for every waypoint. Out of the algebraic solutions obtained, the trajectory generator always chooses the branch that minimises the Euclidean distance with respect to the previous solution set, thereby enforcing continuity across segments.

With a consistent joint sequence in hand the node carries out cubic time-parameterisation that respects the UR5’s limits. The result is wrapped into a `JointTrajectory` message, embedded in a thin `ViaPoints` envelope, and forwarded. A watchdog running in `table_avoidance.cpp` remains active in parallel, examining the pose of the end effector at 100 Hz and publishing an emergency shutdown flag whenever the tool threatens to collide with the workbench.

5 Interface Package

The final transfer of authority takes place in the interface package. `trajectory_execution.cpp` listens continuously for new `ViaPoints`. Upon reception it unwraps the enclosed `JointTrajectory`, frames it as a goal for the `scaled_joint_trajectory_controller`, and delivers the goal through the ROS 2 action interface. The node then waits in an event loop, relaying feedback to the planner and ultimately emitting a Boolean flag that indicates success or failure. Two additional services, implemented in `gripper_service.cpp`, translate simple open/close requests into UR Script commands that are streamed to the gripper’s controller via a raw TCP connection on port 30002. A stand-alone application, `publish_trajectory_node.cpp`, exists purely for calibration and diagnosis; it is not involved during standard operation.

6 Conclusion

The robot orchestrates perception, planning, motion generation, and execution in a tight yet transparent loop that resembles a classical sense-plan-act architecture. Data always flow forward: images beget poses, poses beget trajectories, and trajectories translate into physical motion. In practice the system handles cluttered workspaces with millimetre accuracy and cycle times well below one second per object. Remaining work focuses on eliminating the clock-synchronisation pitfall uncovered at the controller interface and on enhancing trajectory smoothness through minimum-jerk profiles. Beyond those improvements the modular design leaves ample room for deeper perception models or compliant control strategies without disrupting the established workflow.